INTERNSHIP RECOS

Low-Level Design Document (LLD)
Condensed implementation design based on the supplied HLD

1. INTRODUCTION AND SYSTEM OVERVIEW

Internship Recos is a web application that collects internship listings from public external sources using scheduled Apify actors. The backend stores raw results, converts them into a common format, validates listings, removes duplicates, and publishes suitable opportunities for students. Students can create profiles, search and filter internships, and save interesting listings. Administrators manage sources and review flagged listings.

1.1 Objectives

• Collect internship listings automatically on a schedule.

• Identify incomplete, suspicious, duplicate, expired, or blocked listings.

• Provide students with a single searchable catalogue of validated opportunities.

• Allow administrators to manage sources, review flagged listings, and monitor scraping runs.

1.2 Scope and assumptions

The platform links students to the original application URL; applications are not submitted inside Internship Recos. The system uses public data and source-specific Apify actors. Scraping must respect each source's terms. Custom machine-learning model training, payments, in-app chat, resume tools, and an employer portal are outside the current scope.

1.3 Technology stack

Layer	Technology / responsibility
Frontend	React, Vite, Tailwind CSS; student web app/PWA and admin dashboard
Backend API	Python FastAPI; authentication, search, profile, saved list, admin endpoints
Scraping	Apify actors, schedules, datasets, and webhook callbacks
Background jobs	Celery workers with Redis queue and rate limiting
Database	PostgreSQL for users, listings, validation, run history, and search

1.4 Main user roles

• Student: register/login, maintain profile, search/filter listings, view details, save or remove bookmarks.

• Administrator: configure sources and schedules, inspect runs and errors, review flagged listings, manage blocklist rules.

2. MODULE DESIGN AND WORKFLOWS

2.1 Authentication and access control

The authentication module registers users, verifies login credentials, and issues a JWT after successful login. Protected endpoints validate the token and role. Student accounts can access student features; administrator endpoints require the Admin role. Passwords are stored as secure hashes, never as plain text.

2.2 Student profile and search

• Profile fields include skills, interests, preferred locations, and minimum stipend.

• The search API returns only listings whose status is Published.

• Filters may include keyword/title, company, location, work mode, role/skills, and stipend.

• The detail page shows the listing description, company, location, stipend, deadline, source, and original apply URL.

• Saved internships are linked to the student account and can be listed or removed.

2.3 Scraping and ingestion workflow

1. Each configured source has an active flag and a cron schedule. An administrator may also start a manual run.

2. Apify runs the selected actor and stores scraped items in its dataset.

3. Apify calls the backend webhook with the run identifier and shared secret.

4. The webhook verifies the request, records the run, and queues a background job. Repeated callbacks for the same run ID must not create duplicate jobs.

5. The worker fetches the dataset and stores each raw item with its JSON payload and content hash.

6. The normaliser maps source-specific fields into the unified internship format.

7. The validator applies hard and soft rules; results and reason codes are recorded.

8. Duplicate records are linked to one canonical internship. Clean records are published; flagged records go to admin review.

2.4 Maintenance workflow

A daily maintenance job expires listings whose application deadline has passed or that have not been seen in two consecutive successful runs. Failed or partial runs must not cause listings to expire. If more than 20% of a run fails to process, the source is marked unhealthy and the administrator is alerted.

3. NORMALISATION, VALIDATION AND STATUS RULES

3.1 Normalisation rules

Input fields	Processing rule
Title / company	Clean text; map role category; canonicalise company name and domain.
Location / work mode	Map city aliases; use onsite, hybrid, or remote.
Stipend / duration	Parse minimum and maximum values, currency (INR by default), and duration in months.
Dates	Convert posted, start, and deadline dates to standard UTC dates; resolve relative dates where possible.
Skills / description	Strip HTML, match skills against a curated dictionary, and preserve original raw payload.
Apply URL / fingerprint	Remove tracking parameters; create a fingerprint from company, title, city, and start month.

3.2 Validation rules

Rule type	Checks	Result
Hard	Required title, company, apply URL, and location or work mode are present; URL is well formed.	Reject
Hard	Listing requests registration fees, deposits, or payment to join.	Reject
Hard	Company/domain matches a blocklist rule.	Reject
Hard	Application deadline is in the past.	Expire
Soft	Apply URL is unreachable or shortened; company website/domain does not match.	Flag for review
Soft	Stipend looks implausible, description is generic, or contact is only via messaging/free-mail.	Flag for review

3.3 Listing status handling

• Published: passes hard checks and has no unresolved flags; visible to students.

• Needs Review: one or more soft-rule flags; hidden until an administrator reviews it.

• Rejected: hard-rule failure; retained for audit but not shown in search.

• Expired: deadline passed or listing was not seen in two consecutive successful runs; hidden from search.

• Run Failed/Partial: retry with backoff up to three times; partial runs do not trigger expiry.

Every validation result stores the rule code, pass/fail outcome, severity, and reason. Status changes are recorded in status_history for audit.

4. DATABASE DESIGN

PostgreSQL is the system of record. Tables are designed around the entities in the HLD; primary keys (PK), foreign keys (FK), and unique constraints (UQ) maintain relationships and prevent duplicate data.

Table	Important fields and purpose
users	id (PK), email (UQ), password_hash, role — login and access role.
user_profiles	user_id (PK/FK), skills, interests, preferred_locations, min_stipend — student preferences.
sources	id (PK), name (UQ), apify_actor_id, schedule_cron, is_active — configured source.
scrape_runs	id (PK), source_id (FK), apify_run_id (UQ), status, started_at, items_fetched — execution tracking.
raw_listings	id (PK), run_id (FK), source_id (FK), payload JSONB, content_hash, fetched_at — original scraped data.
companies	id (PK), canonical_name, domain (UQ), is_blocklisted — canonical company records.
internships	id (PK), company_id (FK), title, city, work_mode, stipend_min/max, apply_url, fingerprint (UQ), status, apply_by, last_seen_at.
internship_sources	internship_id + source_id (composite PK/FK), raw_listing_id (FK), source_url — source links for merged listings.
validation_results	id (PK), internship_id (FK), raw_listing_id (FK), rule_code, passed, severity — rule outcomes.
status_history	id (PK), internship_id (FK), from_status, to_status, actor_type, created_at — audit trail.
saved_internships	user_id + internship_id (composite PK/FK), saved_at — student bookmarks.
blocklist_rules	id (PK), type, pattern, reason — blocked companies/domains/keywords.

4.1 Key relationships

• One source can have many scrape runs; each run can produce many raw listings.

• A company can have many internships; an internship can be found on multiple sources through internship_sources.

• An internship can have multiple validation results and status-history entries.

• A user has one profile and can save many internships; an internship can be saved by many users.

Raw payloads and audit history should be retained rather than overwritten. Add indexes for internship status, deadline, city, company, fingerprint, and PostgreSQL full-text search fields.

5. API DESIGN, SECURITY AND ERROR HANDLING

5.1 API endpoint summary

Route	Method / purpose	Access
/auth/register	POST — create account	Public
/auth/login	POST — verify credentials and return JWT	Public
/internships	GET — search/filter Published listings	Student
/internships/{id}	GET — listing details	Student
/profile	GET / PUT — view or update profile	Student
/saved	GET / POST / DELETE — list, save, or remove bookmark	Student
/ingest/apify-webhook	POST — verify webhook, record run, enqueue job	Shared secret
/admin/sources	GET / POST / PUT — manage sources and schedules	Admin
/admin/runs	GET / POST — view run history or trigger a run	Admin
/admin/review	GET / POST — review, publish, or reject flagged listings	Admin
/admin/blocklist	GET / POST / DELETE — manage blocklist rules	Admin

5.2 Security requirements

• Use HTTPS for all client/API traffic.

• Hash passwords securely and issue short-lived JWTs; check role permissions on every protected endpoint.

• Protect the Apify webhook with a shared secret and make processing idempotent by run ID.

• Store API keys and database credentials in environment variables or a secrets manager.

• Validate inputs, parameterise database queries, apply rate limits, and avoid exposing internal errors to users.

• Use only permitted public data and link back to the original listing source.

5.3 Errors and recovery

Problem	Handling
Apify timeout / failed run	Record error, retry with exponential backoff up to three attempts, and show run status to admin.
Invalid scraped item	Keep raw payload; record mapper/validation error; skip publishing that item.
Duplicate webhook	Recognise existing Apify run ID and avoid duplicate processing.
Source layout change	Record mapper errors; alert admin if more than 20% of items fail; allow raw data to be reprocessed.
Database/API failure	Return a safe error response, log a correlation ID, and avoid exposing secrets.

6. TESTING AND NON-FUNCTIONAL REQUIREMENTS

6.1 Test cases

ID	Test scenario	Expected result
TC-01	Register with valid details	Account created; password stored as hash.
TC-02	Login with wrong password	Authentication rejected.
TC-03	Search for a keyword/location	Only matching Published listings returned.
TC-04	Submit a listing missing required fields	Rejected with a validation reason.
TC-05	Submit a fee-demanding listing	Rejected and retained for audit.
TC-06	Same listing appears on two sources	One canonical internship; both source links retained.
TC-07	Send the same webhook twice	Only one run job is queued.
TC-08	Run is partial or fails	Retry/alert occurs; listings are not expired by that run.
TC-09	Student accesses admin endpoint	Request denied.
TC-10	Student saves/removes listing	Bookmark is added/removed for that account.

6.2 Non-functional requirements

• Reliability: background jobs, bounded retries, idempotent ingestion, and run monitoring.

• Performance: indexed search fields and pagination for listing results.

• Security: role-based access, hashed passwords, secret management, input validation, and HTTPS.

• Maintainability: source-specific mappers, configurable rules, and recorded validation reasons.

• Auditability: retain raw payloads, validation results, run history, and status changes.

• Compliance and cost: use permitted public data, configure item caps, and schedule each source appropriately.

6.3 Design limitations and assumptions

This LLD translates the supplied HLD into implementation-level module, database, API, and test details. Exact Apify actors, source-specific fields, deployment settings, and final validation thresholds must be selected during implementation after reviewing each source’s terms. The platform flags suspicious listings; it cannot guarantee that every published internship is genuine.